CS2 Dataset Overview and Exploratory Analysis

Round-level, round-state, and enriched data

Table of contents

1 Setup **2**

2 1. Dataset structure and quality **4**
- 2.1 Overview of the datasets . . . 4
- 2.2 Column names and data types . . . 5
- 2.3 Columns with missing values . . . 7
- 2.4 Maps, teams, and dataset coverage . . . 8

3 2. Round-level analysis (`rounds`) **9**
- 3.1 Round keys and match length . . . 9
- 3.2 Round summary by map . . . 10
- 3.3 Round duration distribution . . . 11
- 3.4 Round wins by side and map . . . 12
- 3.5 Win conditions by map . . . 13
- 3.6 Team performance by map and side . . . 15
- 3.7 Team round-win distribution by map . . . 18

4 3. Round-state analysis (`round_states`) **20**
- 4.1 Snapshot coverage and sampling . . . 20
- 4.2 Alive-player advantage heatmap . . . 21
- 4.3 Round-end alive players and health . . . 22
- 4.4 Round-end alive-count distribution . . . 24

5 4. Enriched round-state analysis (`round_states_enriched`) **26**
- 5.1 Enriched data checks . . . 26
- 5.2 Within-round consistency check . . . 27
- 5.3 Bomb plant rates and plant-site distribution . . . 29
- 5.4 T win rate with and without a plant . . . 31
- 5.5 Time until plant and post-plant duration . . . 33

5.6 CT win rate by buy-type matchup 35
5.7 Round outcome versus alive-player advantage at 30 seconds 37
5.8 Equipment, money, health, and armor advantage at 30 seconds 38

6 5. Interpretation notes and limitations 40

1 Setup

This report combines the dataset overview, round-level descriptive analysis, team/map/side comparisons, round-state analysis, and bomb/economy/player-state analyses into one logically ordered PDF report. Summary tables appear in the report and are also exported as CSV files to `../data/analysis`. Figures are saved as PDF files under `../report/figures/cs2-combined_analysis`; CSV tables are saved under `../data/analysis/cs2-combined_analysis`. These paths assume this QMD is stored in `cs2-rop/R`.

How to read the tables: Some PDF tables are shortened to fit the page. Their complete versions are saved as CSV files in `../data/analysis/cs2-combined_analysis/`. Where a table is abbreviated, a note above it identifies the matching CSV filename. Figures are saved as PDFs in `../report/figures/cs2-combined_analysis/`.

```
library(tidyverse)
library(arrow)
library(knitr)
library(scales)

rounds_file <- "../data/analysis/rounds.parquet"
states_file <- "../data/analysis/round_states.parquet"
enriched_file <- "../data/analysis/round_states_enriched.parquet"
figures_dir <- "../report/figures/cs2-combined_analysis"
tables_dir <- "../data/analysis/cs2-combined_analysis"
dir.create(figures_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(tables_dir, showWarnings = FALSE, recursive = TRUE)

save_csv <- function(data, filename) {
  readr::write_csv(data, file.path(tables_dir, filename))
}

rounds <- read_parquet(rounds_file) |> as_tibble()
round_states <- read_parquet(states_file) |> as_tibble()
round_states_enriched <- read_parquet(enriched_file) |> as_tibble()
```

```r
map_label <- function(x) tools::toTitleCase(sub("^de_", "", as.character(x)))
rounds <- rounds |> mutate(map = map_label(map))
round_states <- round_states |> mutate(map = map_label(map))
round_states_enriched <- round_states_enriched |> mutate(map = map_label(map))

show_table <- function(x, digits = 2) {
  # Use LaTeX tables for PDF output. Long tables can continue across pages.
  knitr::kable(
    x, digits = digits, format = "latex", booktabs = TRUE,
    longtable = nrow(x) > 25, row.names = FALSE
  )
}

# State where readers can find the complete CSV whenever a PDF table is abbreviated.
pdf_table_note <- function(csv_name) {
  cat(
    "**Note:** This PDF table is abbreviated for readability. The complete table is available
    csv_name,
    "`. Other CSV exports are in `../data/analysis/cs2-combined_analysis/`.",
    sep = ""
  )
}
save_pdf <- function(plot, filename, width = 9, height = 6) {
  ggsave(file.path(figures_dir, filename), plot = plot,
         width = width, height = height, device = "pdf")
}
theme_set(theme_minimal(base_size = 11))

# Consistent earthy palette for categorical plots.
# CT/T colours remain fixed across every side-based plot.
earth_palette <- c(
  "bomb_defused" = "#B86F5C",
  "ct_win" = "#A68B2D",
  "NA" = "#8C9A7A",
  "target_bombed" = "#5EAAA8",
  "target_saved" = "#6F8FB5",
  "terrorists_win" = "#A879B8",
  "Dust2" = "#B86F5C",
  "Mirage" = "#5E8C8A",
  "CT" = "#4a7fb5",
  "T" = "#d9a03f",
  "Counter-Terrorist" = "#4a7fb5",
```

```
  "Terrorist" = "#d9a03f"
)

pct <- scales::label_percent(accuracy = 1, scale = 1)
```

2 1. Dataset structure and quality

2.1 Overview of the datasets

`rounds` has one record per round; `round_states` has repeated snapshots over time within a round. `round_states_enriched` attaches selected round-level fields to every snapshot. It is a merged, deduplicated-by-column-selection analytical table: it keeps the snapshot rows and adds round context, making it useful for examining how round-level outcomes relate to the evolving state of a round. It is not one row per round.

```
dataset_overview <- tibble(
  dataset = c("rounds", "round_states", "round_states_enriched"),
  rows = c(nrow(rounds), nrow(round_states), nrow(round_states_enriched)),
  columns = c(ncol(rounds), ncol(round_states), ncol(round_states_enriched)),
  missing_cells = c(sum(is.na(rounds)), sum(is.na(round_states)),
                    sum(is.na(round_states_enriched))),
  unique_matches = c(n_distinct(rounds$match_id),
                     n_distinct(round_states$match_id),
                     n_distinct(round_states_enriched$match_id)),
  unique_rounds = c(
    nrow(distinct(rounds, match_id, map, round_num)),
    nrow(distinct(round_states, match_id, map, round_num)),
    nrow(distinct(round_states_enriched, match_id, map, round_num))
  )
)
show_table(dataset_overview)
```

dataset	rows	columns	missing_cells	unique_matches	unique_rounds
rounds	1257	26	1288	60	1257
round_states	116201	45	195710	60	1257
round_states_enriched	116201	59	250917	60	1257

```
save_csv(dataset_overview, "qmd_dataset_overview.csv")
```

2.2 Column names and data types

Only the two original datasets are listed here. If a column appears in both datasets, its `dataset` entry says **Both**. The enriched dataset is derived from these two tables and is described above rather than listing a third, largely repetitive column inventory.

```
round_cols <- tibble(
  column = names(rounds),
  rounds_type = map_chr(rounds, ~ paste(class(.x), collapse = "/"))
)
state_cols <- tibble(
  column = names(round_states),
  round_states_type = map_chr(round_states, ~ paste(class(.x), collapse = "/"))
)
column_overview <- full_join(round_cols, state_cols, by = "column") |>
  mutate(
    dataset = case_when(
      !is.na(rounds_type) & !is.na(round_states_type) ~ "Both rounds and round_states",
      !is.na(rounds_type) ~ "rounds",
      TRUE ~ "round_states"
    ),
    data_type = coalesce(rounds_type, round_states_type)
  ) |>
  select(column, dataset, data_type) |>
  arrange(dataset, column)
show_table(column_overview)
```

column	dataset	data_type
bomb_planted	Both rounds and round_states	logical
ct_score_before	Both rounds and round_states	integer
ct_team	Both rounds and round_states	character
ct_win	Both rounds and round_states	logical
map	Both rounds and round_states	character
match_id	Both rounds and round_states	character
part	Both rounds and round_states	integer
plant_site	Both rounds and round_states	character
round_num	Both rounds and round_states	integer
t_score_before	Both rounds and round_states	integer
t_team	Both rounds and round_states	character
winner_side	Both rounds and round_states	character

bomb_carried	round_states	logical
ct_alive	round_states	integer
ct_armor	round_states	integer
ct_awps	round_states	integer
ct_defusers	round_states	integer
ct_equipment	round_states	integer
ct_flashes	round_states	integer
ct_he	round_states	integer
ct_helmets	round_states	integer
ct_hp	round_states	integer
ct_molotovs	round_states	integer
ct_money	round_states	integer
ct_rifles	round_states	integer
ct_smgs	round_states	integer
ct_smokes	round_states	integer
seconds	round_states	numeric
seconds_since_plant	round_states	numeric
t_alive	round_states	integer
t_armor	round_states	integer
t_awps	round_states	integer
t_defusers	round_states	integer
t_equipment	round_states	integer
t_flashes	round_states	integer
t_he	round_states	integer
t_helmets	round_states	integer
t_hp	round_states	integer
t_molotovs	round_states	integer
t_money	round_states	integer
t_rifles	round_states	integer
t_smgs	round_states	integer
t_smokes	round_states	integer
tick	round_states	integer
time_remaining_s	round_states	numeric
ct_buy_type	rounds	character
ct_equipment_freeze_end	rounds	integer
ct_won_previous_round	rounds	logical
end_tick	rounds	integer
first_death_side	rounds	character
freeze_end_tick	rounds	integer

is_overtime	rounds	logical
is_pistol_round	rounds	logical
plant_tick	rounds	integer
reason_name	rounds	character
round_duration_s	rounds	numeric
t_buy_type	rounds	character
t_equipment_freeze_end	rounds	integer
winner_team	rounds	character

```
save_csv(column_overview, "qmd_column_overview.csv")
```

2.3 Columns with missing values

Only columns that contain at least one missing value are shown. Missingness is not automatically an error: for example, planting-related fields are naturally unavailable in rounds where the bomb was not planted.

```
missing_by_dataset <- function(data, dataset_name) {
  tibble(
    dataset = dataset_name,
    column = names(data),
    missing_n = map_int(data, ~ sum(is.na(.x))),
    missing_pct = map_dbl(data, ~ 100 * mean(is.na(.x)))
  ) |>
    filter(missing_n > 0) |>
    arrange(desc(missing_n))
}
missing_original <- bind_rows(
  missing_by_dataset(rounds, "rounds"),
  missing_by_dataset(round_states, "round_states")
) |>
  filter(missing_n > 0)
show_table(missing_original)
```

dataset	column	missing_n	missing_pct
rounds	plant_site	596	47.41
rounds	plant_tick	596	47.41
rounds	ct_won_previous_round	60	4.77
rounds	reason_name	12	0.95
rounds	ct_buy_type	11	0.88
rounds	t_buy_type	11	0.88
rounds	first_death_side	2	0.16
round_states	seconds_since_plant	97855	84.21
round_states	plant_site	97855	84.21

```
save_csv(missing_original, "qmd_missing_values_original_datasets.csv")
```

2.4 Maps, teams, and dataset coverage

```
map_overview <- rounds |>
  group_by(map) |>
  summarise(matches = n_distinct(match_id), rounds = n(), .groups = "drop")
show_table(map_overview)
```

map	matches	rounds
Dust2	30	628
Mirage	30	629

```
save_csv(map_overview, "qmd_map_overview.csv")

team_overview <- bind_rows(
  rounds |> transmute(team = ct_team),
  rounds |> transmute(team = t_team)
) |>
  filter(!is.na(team), team != "") |>
  distinct(team) |>
  arrange(team)
show_table(team_overview)
```

team
9z
Aurora
FURIA
FUT
FaZe
Falcons
G2
Legacy
MOUZ
NAVI
Spirit
Vitality

```
save_csv(team_overview, "qmd_team_overview.csv")
```

3 2. Round-level analysis (rounds)

3.1 Round keys and match length

```
round_key_check <- rounds |>
  count(match_id, map, round_num, name = "rows_per_round") |>
  filter(rows_per_round > 1)
tibble(duplicate_round_keys = nrow(round_key_check)) |> show_table()
```

duplicate_round_keys
0

```
if (nrow(round_key_check) > 0) show_table(round_key_check)

rounds_per_match <- rounds |>
  distinct(match_id, map, round_num) |>
  count(match_id, map, name = "rounds_in_match")
show_table(rounds_per_match |> summarise(
  matches = n(), min_rounds = min(rounds_in_match),
  q1_rounds = quantile(rounds_in_match, .25),
  median_rounds = median(rounds_in_match),
```

```
  mean_rounds = mean(rounds_in_match),
  q3_rounds = quantile(rounds_in_match, .75),
  max_rounds = max(rounds_in_match)
))
```

matches	min_rounds	q1_rounds	median_rounds	mean_rounds	q3_rounds	max_rounds
60	14	18.75	21	20.95	22.25	35

3.2 Round summary by map

```
# Build the summary in this chunk before using it.
rounds_by_map <- rounds |>
  group_by(map) |>
  summarise(
    matches = n_distinct(match_id),
    rounds = n(),
    pistol_rounds = sum(is_pistol_round %in% TRUE, na.rm = TRUE),
    overtime_rounds = sum(is_overtime %in% TRUE, na.rm = TRUE),
    planted_rounds = sum(bomb_planted %in% TRUE, na.rm = TRUE),
    planted_pct = 100 * mean(bomb_planted %in% TRUE, na.rm = TRUE),
    mean_duration_s = mean(round_duration_s, na.rm = TRUE),
    median_duration_s = median(round_duration_s, na.rm = TRUE),
    ct_round_wins = sum(ct_win %in% TRUE, na.rm = TRUE),
    t_round_wins = sum(ct_win %in% FALSE, na.rm = TRUE),
    .groups = "drop"
  )

rounds_by_map_pdf <- rounds_by_map |>
  select(
    map, matches, rounds, planted_pct,
    mean_duration_s, median_duration_s
  ) |>
  rename(
    Map = map,
    Matches = matches,
    Rounds = rounds,
    `Bomb planted (%)` = planted_pct,
    `Mean duration (s)` = mean_duration_s,
    `Median duration (s)` = median_duration_s
```

```
  )

cat(
  "**Note:** This PDF table is shortened to fit the page. The complete table is saved as ",
  "`../data/analysis/cs2-combined_analysis/qmd_rounds_by_map.csv`.",
  sep = ""
)
```

Note: This PDF table is shortened to fit the page. The complete table is saved as `../dat
combined_analysis/qmd_rounds_by_map.csv`.

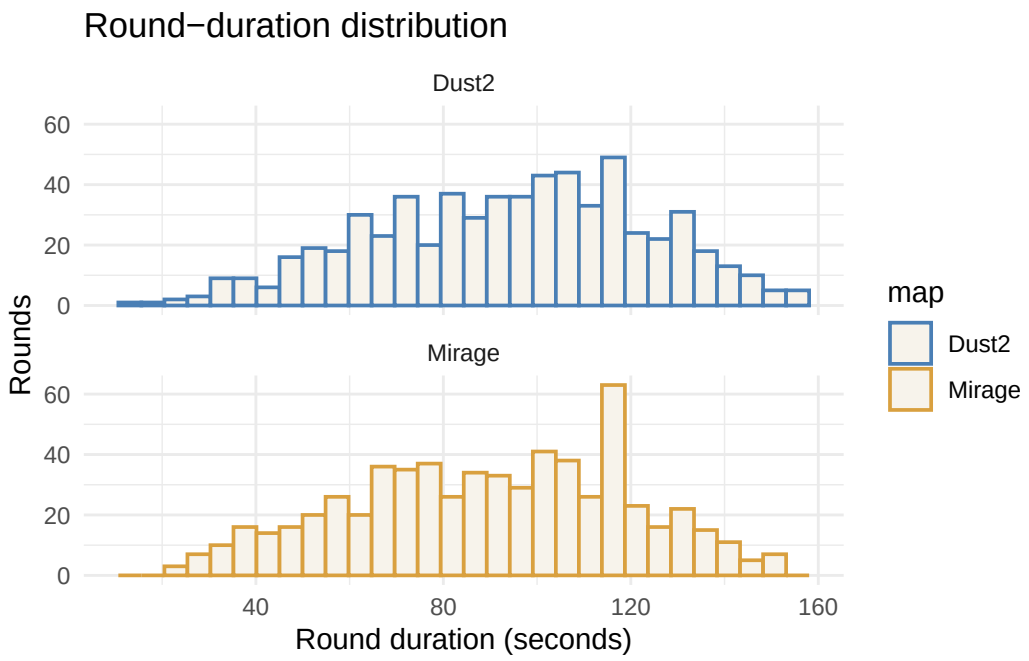
```
show_table(rounds_by_map_pdf)
```

Map	Matches	Rounds	Bomb planted (%)	Mean duration (s)	Median duration (s)
Dust2	30	628	55.57	94.20	96.73
Mirage	30	629	49.60	89.75	91.30

```
# Save all columns, not just the compact PDF selection.
save_csv(rounds_by_map, "qmd_rounds_by_map.csv")
```

3.3 Round duration distribution

```
p_duration <- ggplot(rounds, aes(x = round_duration_s, colour = map)) +
  geom_histogram(bins = 30, fill = "#f7f3eb", linewidth = 0.65,
                 position = "identity") +
  scale_colour_manual(values = c(Dust2 = "#4a7fb5", Mirage = "#d9a03f")) +
  facet_wrap(~map, ncol = 1) +
  labs(title = "Round-duration distribution", x = "Round duration (seconds)", y = "Rounds")
p_duration
```

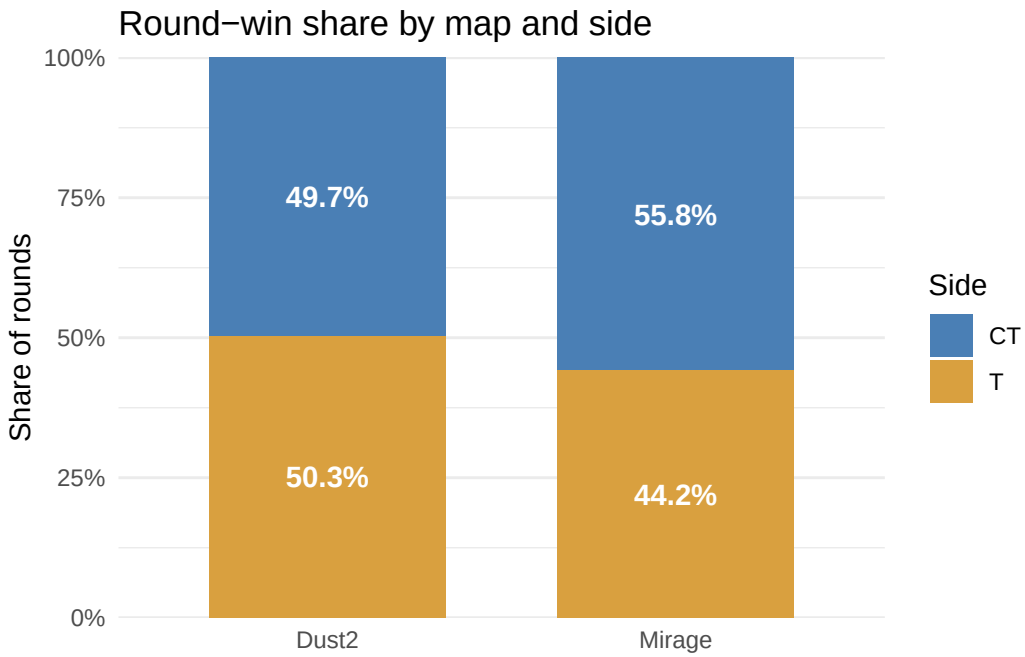



```
save_pdf(p_duration, "01_round_duration_distribution.pdf", 8, 7)
```

3.4 Round wins by side and map

Percentages are calculated within each map, so each map's stacked bar sums to 100%.

```
side_distribution <- rounds |>
  group_by(map) |>
  summarise(CT = mean(ct_win, na.rm = TRUE) * 100,
            T = mean(!ct_win, na.rm = TRUE) * 100, .groups = "drop") |>
  pivot_longer(c(CT, T), names_to = "side", values_to = "percentage")
p_side <- ggplot(side_distribution, aes(map, percentage, fill = side)) +
  geom_col(width = .68) +
  geom_text(aes(label = sprintf("%.1f%%", percentage)),
            position = position_stack(vjust = .5), colour = "white", fontface = "bold") +
  scale_fill_manual(values = c(CT = "#4a7fb5", T = "#d9a03f")) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
                     expand = c(0, 0)) +
  scale_fill_manual(values = c(T = "#d9a03f", CT = "#4a7fb5")) +
  labs(title = "Round-win share by map and side", x = NULL, y = "Share of rounds", fill = "Si
p_side
```




```
save_pdf(p_side, "02_side_win_share_by_map.pdf", 7, 5)
```

3.5 Win conditions by map

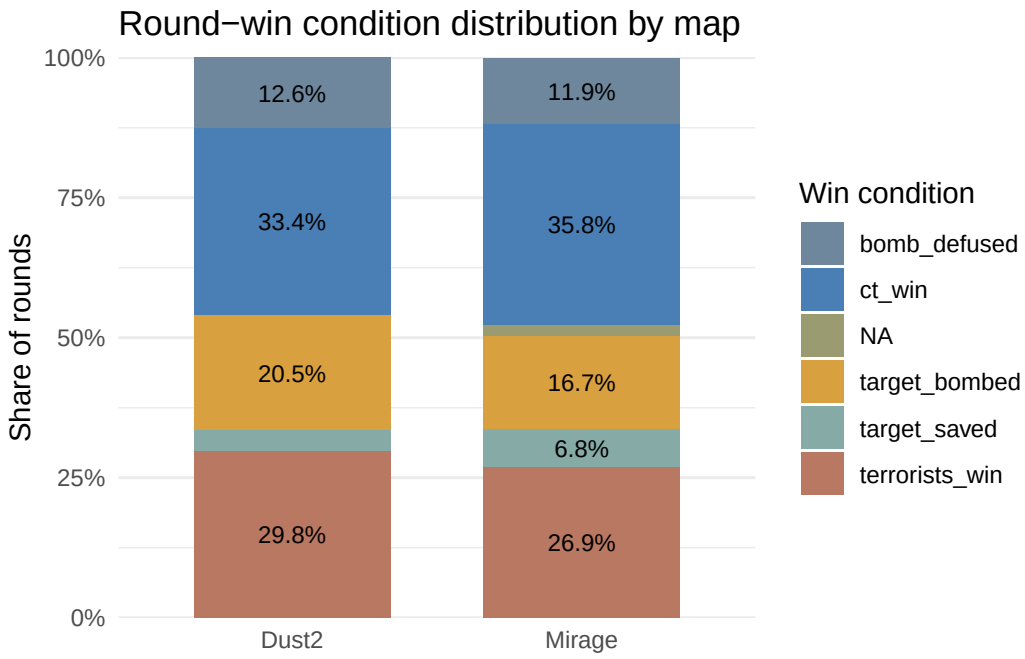
Missing `reason_name` values remain missing and are displayed as `NA`; they are not relabelled as a specific win condition.

```
win_condition_summary <- rounds |>
  group_by(map, reason_name) |>
  summarise(rounds = n(), .groups = "drop_last") |>
  mutate(pct_of_map_rounds = 100 * rounds / sum(rounds)) |>
  ungroup() |>
  mutate(reason_label = if_else(is.na(reason_name), "NA", as.character(reason_name)))
show_table(win_condition_summary |> select(map, reason_label, rounds, pct_of_map_rounds))
```

map	reason_label	rounds	pct_of_map_rounds
Dust2	bomb_defused	79	12.58
Dust2	ct_win	210	33.44
Dust2	target_bombed	129	20.54
Dust2	target_saved	23	3.66
Dust2	terrorists_win	187	29.78
Mirage	bomb_defused	75	11.92
Mirage	ct_win	225	35.77
Mirage	target_bombed	105	16.69
Mirage	target_saved	43	6.84
Mirage	terrorists_win	169	26.87
Mirage	NA	12	1.91

```
save_csv(win_condition_summary |> select(map, reason_label, rounds, pct_of_map_rounds), "qmd_

condition_palette <- c(
  bomb_defused = "#6f879d",
  ct_win = "#4a7fb5",
  "NA" = "#9a9b70",
  target_bombed = "#d9a03f",
  target_saved = "#86aaa6",
  terrorists_win = "#b97862"
)
p_conditions <- ggplot(win_condition_summary,
                       aes(x = map, y = pct_of_map_rounds, fill = reason_label)) +
  geom_col(width = .68) +
  scale_fill_manual(values = condition_palette, na.value = "#9a9b70") +
  geom_text(aes(label = if_else(pct_of_map_rounds >= 5,
                                sprintf("%.1f%%", pct_of_map_rounds), "")),
            position = position_stack(vjust = .5), size = 3) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
                     expand = c(0, 0)) +
  labs(title = "Round-win condition distribution by map", x = NULL,
       y = "Share of rounds", fill = "Win condition")
p_conditions
```




```
save_pdf(p_conditions, "03_win_condition_distribution.pdf", 9, 6)
```

3.6 Team performance by map and side

Each round contributes one observation for each participating team, assigned to its side for that round.

```
team_side_rounds <- bind_rows(
  rounds |> transmute(match_id, round_num, map, team = ct_team, side = "CT",
                      side_won = ct_win %in% TRUE, winner_team, round_duration_s,
                      is_pistol_round, is_overtime),
  rounds |> transmute(match_id, round_num, map, team = t_team, side = "T",
                      side_won = ct_win %in% FALSE, winner_team, round_duration_s,
                      is_pistol_round, is_overtime)
) |>
  filter(!is.na(team), team != "")
team_map_side_summary <- team_side_rounds |>
  group_by(map, team, side) |>
  summarise(
    matches = n_distinct(match_id), rounds_played = n(),
    rounds_won_on_side = sum(side_won, na.rm = TRUE),
    side_round_win_pct = 100 * mean(side_won, na.rm = TRUE),
```

```
    mean_round_duration_s = mean(round_duration_s, na.rm = TRUE),
    pistol_rounds = sum(is_pistol_round %in% TRUE, na.rm = TRUE),
    overtime_rounds = sum(is_overtime %in% TRUE, na.rm = TRUE),
    .groups = "drop"
  )
# Compact version for the PDF; the full table is preserved in the CSV.
team_map_side_summary_pdf <- team_map_side_summary |>
  arrange(map, team, side) |>
  select(
    map, team, side, matches, rounds_played,
    rounds_won_on_side, side_round_win_pct
  ) |>
  rename(
    `Map` = map,
    `Team` = team,
    `Side` = side,
    `Matches` = matches,
    `Rounds played` = rounds_played,
    `Rounds won` = rounds_won_on_side,
    `Side win rate (%)` = side_round_win_pct
  )
pdf_table_note("qmd_team_map_side_summary.csv")
```

Note: This PDF table is abbreviated for readability. The complete table is available in `../data/analysis/combined_analysis/qmd_team_map_side_summary.csv`. Other CSV exports are in `../data/analysis/combined_analysis/`.

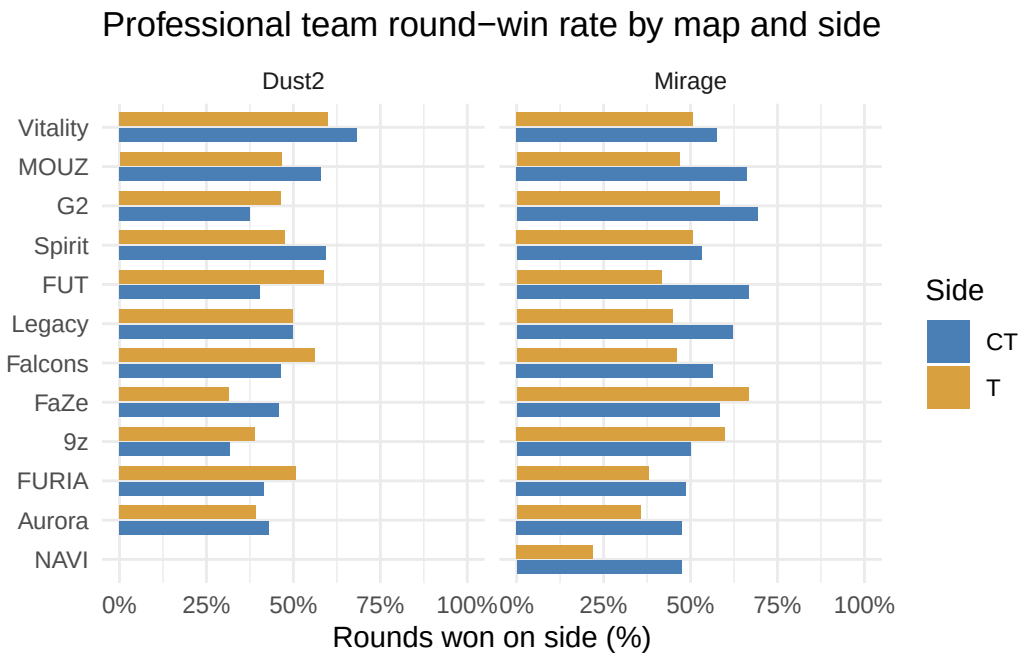
```
show_table(team_map_side_summary_pdf)
```

Map	Team	Side	Matches	Rounds played	Rounds won	Side win rate (%)
Dust2	9z	CT	3	22	7	31.82
Dust2	9z	T	3	36	14	38.89
Dust2	Aurora	CT	2	21	9	42.86
Dust2	Aurora	T	2	23	9	39.13
Dust2	FURIA	CT	8	82	34	41.46
Dust2	FURIA	T	8	77	39	50.65
Dust2	FUT	CT	4	52	21	40.38
Dust2	FUT	T	4	51	30	58.82
Dust2	FaZe	CT	2	24	11	45.83

Dust2	FaZe	T	2	19	6	31.58
Dust2	Falcons	CT	5	56	26	46.43
Dust2	Falcons	T	5	48	27	56.25
Dust2	G2	CT	6	72	27	37.50
Dust2	G2	T	6	43	20	46.51
Dust2	Legacy	CT	7	72	36	50.00
Dust2	Legacy	T	7	74	37	50.00
Dust2	MOUZ	CT	7	88	51	57.95
Dust2	MOUZ	T	7	73	34	46.58
Dust2	Spirit	CT	7	54	32	59.26
Dust2	Spirit	T	7	84	40	47.62
Dust2	Vitality	CT	9	85	58	68.24
Dust2	Vitality	T	9	100	60	60.00
Mirage	9z	CT	1	14	7	50.00
Mirage	9z	T	1	15	9	60.00
Mirage	Aurora	CT	8	82	39	47.56
Mirage	Aurora	T	8	73	26	35.62
Mirage	FURIA	CT	7	82	40	48.78
Mirage	FURIA	T	7	87	33	37.93
Mirage	FUT	CT	1	12	8	66.67
Mirage	FUT	T	1	12	5	41.67
Mirage	FaZe	CT	1	12	7	58.33
Mirage	FaZe	T	1	9	6	66.67
Mirage	Falcons	CT	6	69	39	56.52
Mirage	Falcons	T	6	50	23	46.00
Mirage	G2	CT	2	13	9	69.23
Mirage	G2	T	2	24	14	58.33
Mirage	Legacy	CT	6	58	36	62.07
Mirage	Legacy	T	6	67	30	44.78
Mirage	MOUZ	CT	9	92	61	66.30
Mirage	MOUZ	T	9	111	52	46.85
Mirage	NAVI	CT	4	38	18	47.37
Mirage	NAVI	T	4	41	9	21.95
Mirage	Spirit	CT	7	79	42	53.16
Mirage	Spirit	T	7	63	32	50.79
Mirage	Vitality	CT	8	78	45	57.69
Mirage	Vitality	T	8	77	39	50.65

```r
save_csv(team_map_side_summary |> arrange(map, team, side),
         "qmd_team_map_side_summary.csv")

p_team_side <- ggplot(team_map_side_summary,
                      aes(x = reorder(team, side_round_win_pct),
                          y = side_round_win_pct, fill = side)) +
  geom_col(position = position_dodge(width = .8), width = .7) +
  coord_flip() + facet_wrap(~map) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%")) +
  scale_fill_manual(values = c(T = "#d9a03f", CT = "#4a7fb5")) +
  labs(title = "Professional team round-win rate by map and side",
       x = NULL, y = "Rounds won on side (%)", fill = "Side")
p_team_side
```




```r
save_pdf(p_team_side, "04_team_win_rate_by_map_side.pdf", 12, 9)
```

3.7 Team round-win distribution by map

This is a count of round-level `winner_team` values, not a reconstructed match winner.

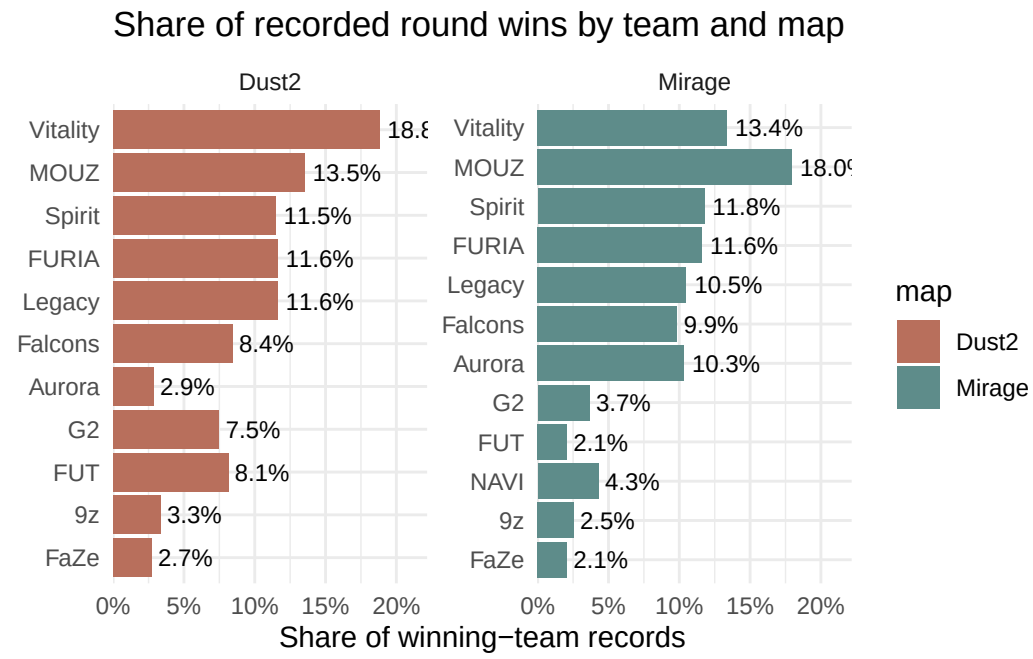
```
team_round_outcomes <- rounds |>
  filter(!is.na(winner_team), winner_team != "") |>
  count(map, winner_team, name = "rounds_won") |>
  group_by(map) |>
  mutate(pct_of_recorded_round_wins = 100 * rounds_won / sum(rounds_won)) |>
  ungroup() |>
  arrange(map, desc(rounds_won))
show_table(team_round_outcomes)
```

map	winner_team	rounds_won	pct_of_recorded_round_wins
Dust2	Vitality	118	18.79
Dust2	MOUZ	85	13.54
Dust2	FURIA	73	11.62
Dust2	Legacy	73	11.62
Dust2	Spirit	72	11.46
Dust2	Falcons	53	8.44
Dust2	FUT	51	8.12
Dust2	G2	47	7.48
Dust2	9z	21	3.34
Dust2	Aurora	18	2.87
Dust2	FaZe	17	2.71
Mirage	MOUZ	113	17.97
Mirage	Vitality	84	13.35
Mirage	Spirit	74	11.76
Mirage	FURIA	73	11.61
Mirage	Legacy	66	10.49
Mirage	Aurora	65	10.33
Mirage	Falcons	62	9.86
Mirage	NAVI	27	4.29
Mirage	G2	23	3.66
Mirage	9z	16	2.54
Mirage	FUT	13	2.07
Mirage	FaZe	13	2.07

```
save_csv(team_round_outcomes, "qmd_team_round_outcomes.csv")

p_team_wins <- ggplot(team_round_outcomes,
                      aes(x = reorder(winner_team, rounds_won),
                          y = pct_of_recorded_round_wins, fill = map)) +
  geom_col() +
```

```
  scale_fill_manual(values = c("Dust2" = "#B86F5C", "Mirage" = "#5E8C8A")) + coord_flip() + t
  geom_text(aes(label = sprintf("%.1f%%", pct_of_recorded_round_wins)),
            hjust = -.12, size = 3) +
  scale_y_continuous(expand = expansion(mult = c(0, .18)),
                     labels = \(x) paste0(x, "%")) +
  labs(title = "Share of recorded round wins by team and map",
       x = NULL, y = "Share of winning-team records")
p_team_wins
```




```
save_pdf(p_team_wins, "05_team_round_win_share.pdf", 12, 9)
```

4 3. Round-state analysis (`round_states`)

4.1 Snapshot coverage and sampling

```
snapshots_per_round <- round_states |>
  count(match_id, map, round_num, name = "snapshots")
show_table(snapshots_per_round |> summarise(
  rounds = n(), min_snapshots = min(snapshots),
```

```
  q1_snapshots = quantile(snapshots, .25),
  median_snapshots = median(snapshots),
  mean_snapshots = mean(snapshots),
  q3_snapshots = quantile(snapshots, .75),
  max_snapshots = max(snapshots)
))
```

rounds	min_snapshots	q1_snapshots	median_snapshots	mean_snapshots	q3_snapshots	max_snap
1257	14	71	95	92.44	115	

```
save_csv(snapshots_per_round, "qmd_snapshots_per_round.csv")
show_table(round_states |> summarise(
  snapshots = n(), min_seconds = min(seconds, na.rm = TRUE),
  median_seconds = median(seconds, na.rm = TRUE),
  max_seconds = max(seconds, na.rm = TRUE)
))
```

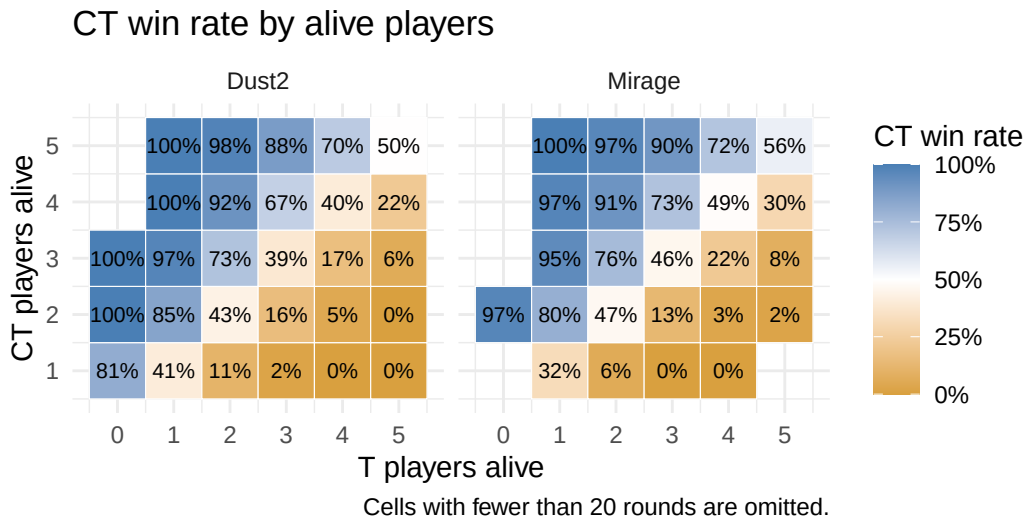
snapshots	min_seconds	median_seconds	max_seconds
116201	0	46	155

4.2 Alive-player advantage heatmap

Each round is counted once per distinct alive-count combination it passed through, so repeated snapshots with the same alive counts do not multiply that combination within a round. Cells with fewer than 20 rounds are omitted.

```
alive <- round_states |>
  distinct(map, match_id, round_num, ct_alive, t_alive, ct_win) |>
  group_by(map, ct_alive, t_alive) |>
  summarise(ct_win_rate = mean(ct_win, na.rm = TRUE), n = n(), .groups = "drop") |>
  filter(n >= 20)
p_alive <- ggplot(alive, aes(t_alive, ct_alive, fill = ct_win_rate)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.0f%%", 100 * ct_win_rate)), size = 2.8) +
  scale_fill_gradient2(low = "#d9a03f", mid = "white", high = "#4a7fb5",
                       midpoint = .5, limits = c(0, 1), labels = percent) +
  scale_x_continuous(breaks = 0:5) + scale_y_continuous(breaks = 0:5) +
  coord_equal() + facet_wrap(~map) +
  labs(title = "CT win rate by alive players", x = "T players alive",
       y = "CT players alive", fill = "CT win rate",
```

```
       caption = "Cells with fewer than 20 rounds are omitted.")
p_alive
```




```
save_pdf(p_alive, "06_alive_player_win_rate_heatmap.pdf", 8, 5)
```

4.3 Round-end alive players and health

For each round, the round-end snapshot is the latest recorded snapshot at or before `end_tick`. Snapshots are periodic, so this is an approximation and may not equal the exact end tick.

```
round_end <- round_states_enriched |>
  filter(!is.na(end_tick), tick <= end_tick) |>
  group_by(match_id, map, round_num) |>
  slice_max(tick, n = 1, with_ties = FALSE) |>
  ungroup()

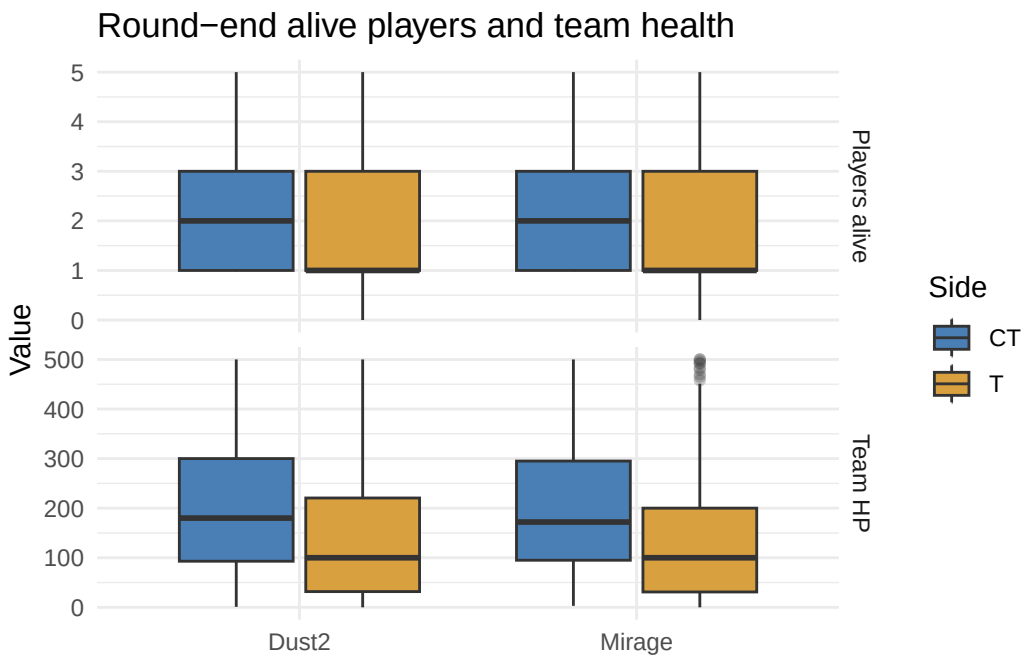
round_end_summary <- round_end |>
  group_by(map) |>
  summarise(
    rounds = n(),
    mean_ct_alive = mean(ct_alive, na.rm = TRUE),
```

```
    median_ct_alive = median(ct_alive, na.rm = TRUE),
    mean_t_alive = mean(t_alive, na.rm = TRUE),
    median_t_alive = median(t_alive, na.rm = TRUE),
    mean_ct_hp = mean(ct_hp, na.rm = TRUE),
    median_ct_hp = median(ct_hp, na.rm = TRUE),
    mean_t_hp = mean(t_hp, na.rm = TRUE),
    median_t_hp = median(t_hp, na.rm = TRUE),
    .groups = "drop"
  )
show_table(round_end_summary)
```

map	rounds	mean_ct_alive	median_ct_alive	mean_t_alive	median_t_alive	mean_ct_hp	me
Dust2	628	2.33	2	1.91	1	190.80	
Mirage	629	2.35	2	1.76	1	189.99	

```
save_csv(round_end_summary, "qmd_round_end_summary.csv")

round_end_long <- round_end |>
  select(map, match_id, round_num, ct_alive, t_alive, ct_hp, t_hp) |>
  pivot_longer(c(ct_alive, t_alive, ct_hp, t_hp),
               names_to = "measure", values_to = "value") |>
  mutate(
    side = if_else(str_starts(measure, "ct_"), "CT", "T"),
    metric = if_else(str_ends(measure, "alive"), "Players alive", "Team HP")
  )
p_round_end <- ggplot(round_end_long, aes(map, value, fill = side)) +
  geom_boxplot(outlier.alpha = .25) +
  scale_fill_manual(values = c(CT = "#4a7fb5", T = "#d9a03f")) +
  facet_grid(metric ~ ., scales = "free_y") +
  scale_fill_manual(values = c(T = "#d9a03f", CT = "#4a7fb5")) +
  labs(title = "Round-end alive players and team health",
       x = NULL, y = "Value", fill = "Side")
p_round_end
```




```
save_pdf(p_round_end, "07_round_end_alive_and_health.pdf", 8, 7)
```

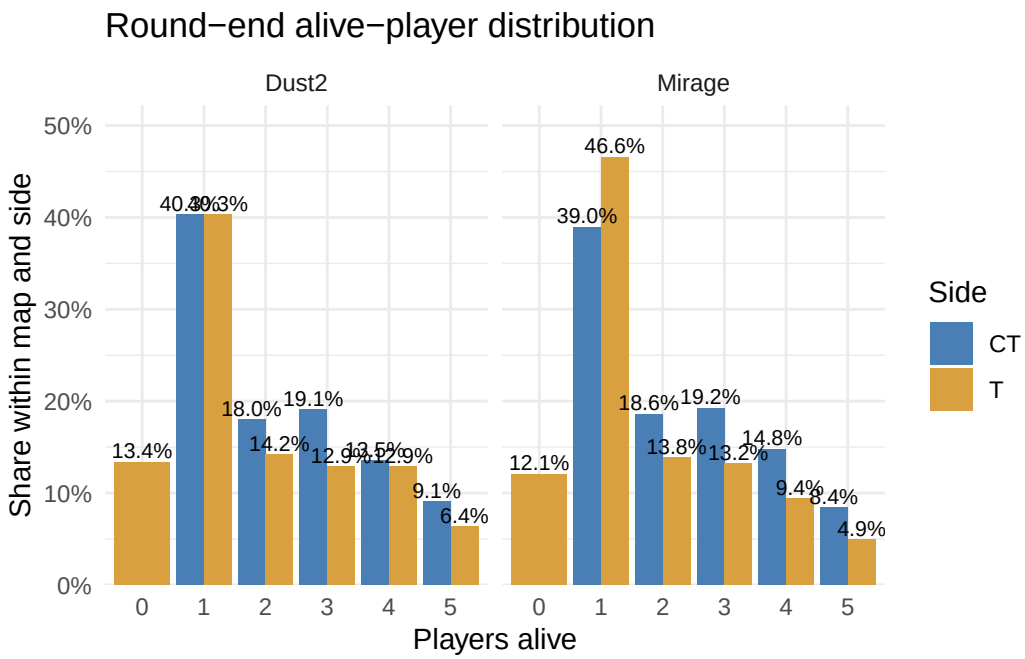
4.4 Round-end alive-count distribution

```
round_end_alive_dist <- round_end |>
  select(map, ct_alive, t_alive) |>
  pivot_longer(c(ct_alive, t_alive), names_to = "side", values_to = "alive") |>
  mutate(side = recode(side, ct_alive = "CT", t_alive = "T")) |>
  count(map, side, alive, name = "rounds") |>
  group_by(map, side) |>
  mutate(pct = 100 * rounds / sum(rounds)) |>
  ungroup()
show_table(round_end_alive_dist)
```

map	side	alive	rounds	pct
Dust2	CT	1	253	40.29
Dust2	CT	2	113	17.99
Dust2	CT	3	120	19.11
Dust2	CT	4	85	13.54
Dust2	CT	5	57	9.08
Dust2	T	0	84	13.38
Dust2	T	1	253	40.29
Dust2	T	2	89	14.17
Dust2	T	3	81	12.90
Dust2	T	4	81	12.90
Dust2	T	5	40	6.37
Mirage	CT	1	245	38.95
Mirage	CT	2	117	18.60
Mirage	CT	3	121	19.24
Mirage	CT	4	93	14.79
Mirage	CT	5	53	8.43
Mirage	T	0	76	12.08
Mirage	T	1	293	46.58
Mirage	T	2	87	13.83
Mirage	T	3	83	13.20
Mirage	T	4	59	9.38
Mirage	T	5	31	4.93

```
save_csv(round_end_alive_dist, "qmd_round_end_alive_distribution.csv")

p_alive_dist <- ggplot(round_end_alive_dist,
                       aes(factor(alive, levels = 0:5), pct, fill = side)) +
  geom_col(position = "dodge") + facet_wrap(~map) +
  scale_fill_manual(values = c(CT = "#4a7fb5", T = "#d9a03f")) +
  geom_text(aes(label = if_else(pct >= 4, sprintf("%.1f%%", pct), "")),
            position = position_dodge(width = .9), vjust = -.25, size = 2.8) +
  scale_y_continuous(labels = \(x) paste0(x, "%"),
                     expand = expansion(mult = c(0, .12))) +
  scale_fill_manual(values = c(T = "#d9a03f", CT = "#4a7fb5")) +
  labs(title = "Round-end alive-player distribution",
       x = "Players alive", y = "Share within map and side", fill = "Side")
p_alive_dist
```




```
save_pdf(p_alive_dist, "08_round_end_alive_distribution.pdf", 9, 5)
```

5 4. Enriched round-state analysis (`round_states_enriched`)

5.1 Enriched data checks

The join should preserve the snapshot row count. Round-level fields repeat on every snapshot belonging to that round; therefore, missing values in fields copied from `rounds` are repeated across all snapshots for the affected round. This repetition is expected and explains why missing-cell counts can be much larger in the enriched table than in the one-row-per-round source.

```
enriched_checks <- tibble(
  dataset = c("round_states", "round_states_enriched"),
  rows = c(nrow(round_states), nrow(round_states_enriched)),
  columns = c(ncol(round_states), ncol(round_states_enriched)),
  distinct_round_keys = c(
    nrow(distinct(round_states, match_id, map, round_num)),
    nrow(distinct(round_states_enriched, match_id, map, round_num))
  )
```

```
)
show_table(enriched_checks)
```

dataset	rows	columns	distinct_round_keys
round_states	116201	45	1257
round_states_enriched	116201	59	1257

```
save_csv(enriched_checks, "qmd_enriched_data_checks.csv")

# Count distinct round keys present in the enriched table but absent from rounds.
round_key_cols <- c("match_id", "map", "round_num")
round_keys <- rounds |> distinct(across(all_of(round_key_cols)))
enriched_keys <- round_states_enriched |> distinct(across(all_of(round_key_cols)))
enriched_rounds_not_in_rounds <- anti_join(enriched_keys, round_keys, by = round_key_cols) |>
  nrow()
join_validation <- tibble(
  check = "enriched_rounds_not_in_rounds",
  value = enriched_rounds_not_in_rounds,
  interpretation = if_else(
    enriched_rounds_not_in_rounds == 0,
    "All enriched round keys match a round in rounds; no unmatched enriched rounds.",
    "Some enriched round keys have no matching round in rounds; inspect the join keys."
  )
)
show_table(join_validation)
```

check	value	interpretation
enriched_rounds_not_in_rounds	0	All enriched round keys match a round in rounds; no unmatched

```
save_csv(join_validation, "qmd_enriched_round_join_validation.csv")
```

5.2 Within-round consistency check

This check counts how many rounds contain more than one distinct non-missing value for each selected field across their snapshots. It is intended to identify fields that should remain constant within a round but unexpectedly vary. `bomb_planted` is the exception: it is a time-varying state, normally `FALSE` before the bomb is planted and `TRUE` afterward. Therefore, the 626 rounds flagged for `bomb_planted` are not automatically errors; they indicate rounds where the snapshots record both states. The other listed fields have zero rounds with multiple values, which is consistent with them remaining constant within each round.

```r
constant_check_fields <- intersect(
  c("winner_side", "winner_team", "reason_name", "is_overtime",
    "is_pistol_round", "bomb_planted", "ct_buy_type", "t_buy_type",
    "first_death_side", "round_duration_s"),
  names(round_states_enriched)
)
round_constant_checks <- map_dfr(constant_check_fields, function(field_name) {
  per_round <- round_states_enriched |>
    group_by(match_id, map, round_num) |>
    summarise(
      distinct_nonmissing_values = n_distinct(.data[[field_name]][!is.na(.data[[field_name]])
      .groups = "drop"
    )
  tibble(
    field = field_name,
    rounds_with_multiple_values = sum(per_round$distinct_nonmissing_values > 1)
  )
}) |>
  mutate(
    interpretation = if_else(
      field == "bomb_planted",
      "Time-varying field: FALSE before plant and TRUE after plant is expected",
      "Expected to remain constant within a round; non-zero counts merit investigation"
    )
  )
show_table(round_constant_checks)
```

field	rounds_with_multiple_values	interpretation
winner_side	0	Expected to remain constant within a round; non-zero
winner_team	0	Expected to remain constant within a round; non-zero
reason_name	0	Expected to remain constant within a round; non-zero
is_overtime	0	Expected to remain constant within a round; non-zero
is_pistol_round	0	Expected to remain constant within a round; non-zero
bomb_planted	626	Time-varying field: FALSE before plant and TRUE aft
ct_buy_type	0	Expected to remain constant within a round; non-zero
t_buy_type	0	Expected to remain constant within a round; non-zero
first_death_side	0	Expected to remain constant within a round; non-zero
round_duration_s	0	Expected to remain constant within a round; non-zero

```r
save_csv(round_constant_checks, "qmd_round_constant_checks.csv")
```

```r
missing_enriched <- missing_by_dataset(round_states_enriched, "round_states_enriched")
show_table(missing_enriched)
```

dataset	column	missing_n	missing_pct
round_states_enriched	seconds_since_plant	97855	84.21
round_states_enriched	plant_site	97855	84.21
round_states_enriched	plant_tick	47215	40.63
round_states_enriched	ct_won_previous_round	4116	3.54
round_states_enriched	reason_name	1263	1.09
round_states_enriched	ct_buy_type	1198	1.03
round_states_enriched	t_buy_type	1198	1.03
round_states_enriched	first_death_side	217	0.19

```r
save_csv(missing_enriched, "qmd_missing_values_enriched.csv")
```

5.3 Bomb plant rates and plant-site distribution

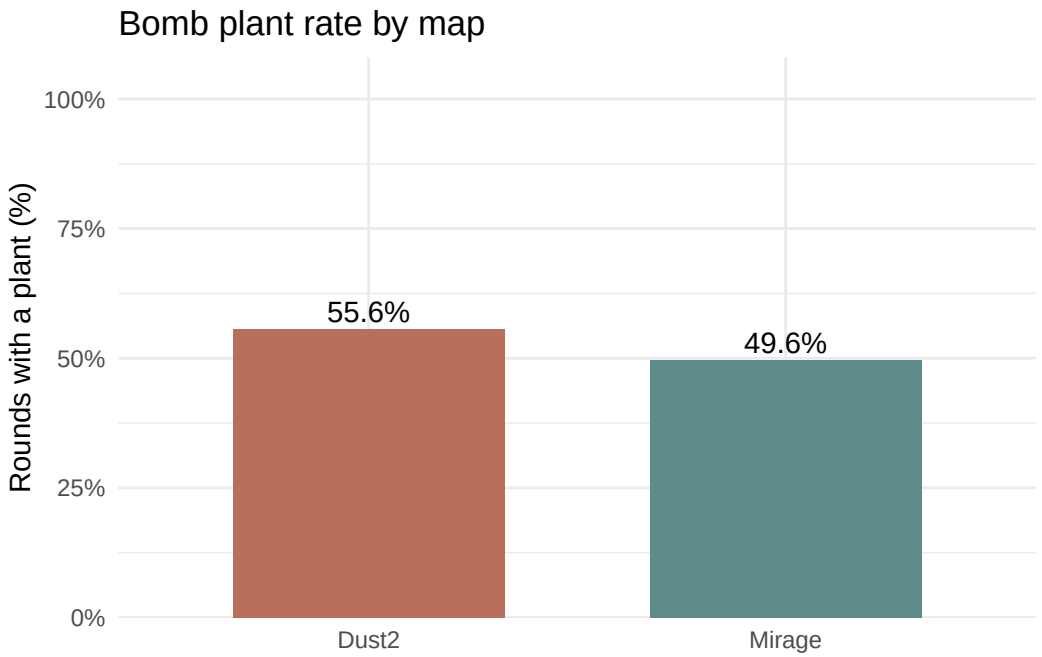
```r
plant_summary <- rounds |>
  group_by(map) |>
  summarise(
    rounds = n(),
    planted_rounds = sum(bomb_planted %in% TRUE, na.rm = TRUE),
    plant_rate_pct = 100 * mean(bomb_planted %in% TRUE, na.rm = TRUE),
    .groups = "drop"
  )
show_table(plant_summary)
```

map	rounds	planted_rounds	plant_rate_pct
Dust2	628	349	55.57
Mirage	629	312	49.60

```r
save_csv(plant_summary, "qmd_bomb_plant_summary.csv")

p_plant <- ggplot(plant_summary, aes(map, plant_rate_pct, fill = map)) +
  geom_col(width = .65) +
  scale_fill_manual(values = c("Dust2" = "#B86F5C", "Mirage" = "#5E8C8A"), guide = "none") +
  geom_text(aes(label = sprintf("%.1f%%", plant_rate_pct)), vjust = -.35) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
```

```
                     expand = expansion(mult = c(0, .08))) +
  labs(title = "Bomb plant rate by map", x = NULL, y = "Rounds with a plant (%)")
p_plant
```



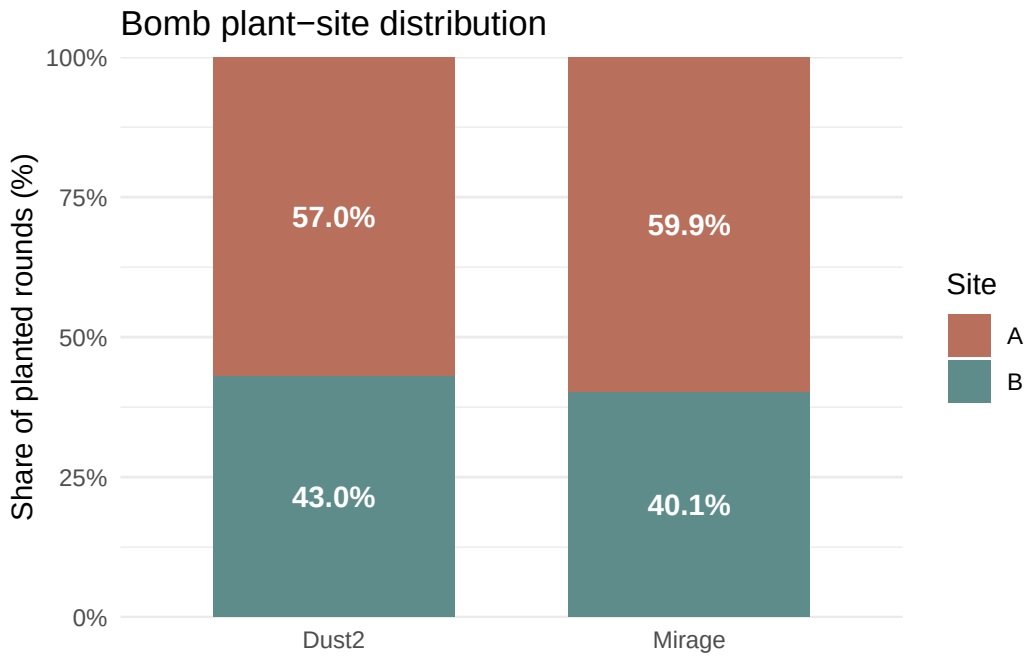

```
save_pdf(p_plant, "09_bomb_plant_rate.pdf", 7, 5)

plant_site <- rounds |>
  filter(bomb_planted %in% TRUE, !is.na(plant_site)) |>
  count(map, plant_site, name = "plants") |>
  group_by(map) |>
  mutate(pct = 100 * plants / sum(plants)) |>
  ungroup()
show_table(plant_site)
```

map	plant_site	plants	pct
Dust2	A	199	57.02
Dust2	B	150	42.98
Mirage	A	187	59.94
Mirage	B	125	40.06

```r
save_csv(plant_site, "qmd_plant_site_distribution.csv")

p_site <- ggplot(plant_site, aes(map, pct, fill = as.character(plant_site))) +
  geom_col(width = .68) +
  geom_text(aes(label = sprintf("%.1f%%", pct)),
            position = position_stack(vjust = .5), colour = "white", fontface = "bold") +
  scale_fill_manual(values = c("A" = "#B86F5C", "B" = "#5E8C8A")) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
                     expand = c(0, 0)) +
  labs(title = "Bomb plant-site distribution", x = NULL,
       y = "Share of planted rounds (%)", fill = "Site")
p_site
```




```r
save_pdf(p_site, "10_bomb_plant_site_distribution.pdf", 7, 5)
```

5.4 T win rate with and without a plant

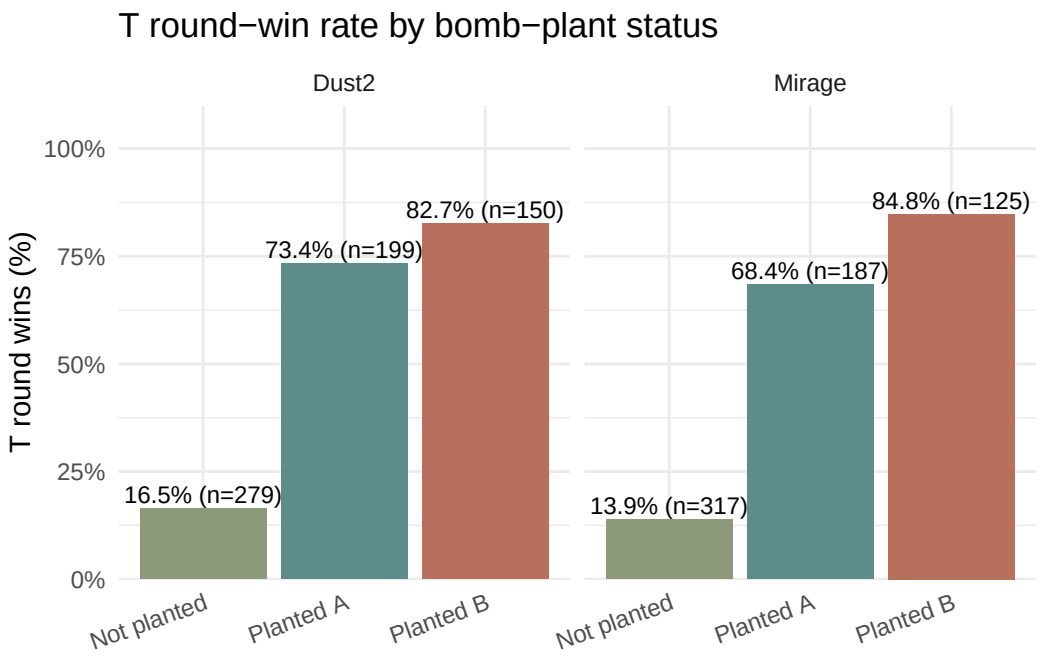
```r
plant_outcomes <- rounds |>
  mutate(plant_status = case_when(
    bomb_planted %in% TRUE ~ paste("Planted", coalesce(as.character(plant_site), "unknown si
```

```
    bomb_planted %in% FALSE ~ "Not planted",
    TRUE ~ "Unknown"
  )) |>
  group_by(map, plant_status) |>
  summarise(
    rounds = n(),
    t_win_pct = 100 * mean(!ct_win, na.rm = TRUE),
    .groups = "drop"
  )
show_table(plant_outcomes)
```

map	plant_status	rounds	t_win_pct
Dust2	Not planted	279	16.49
Dust2	Planted A	199	73.37
Dust2	Planted B	150	82.67
Mirage	Not planted	317	13.88
Mirage	Planted A	187	68.45
Mirage	Planted B	125	84.80

```
save_csv(plant_outcomes, "qmd_plant_outcomes.csv")

p_plant_outcome <- ggplot(plant_outcomes,
                          aes(plant_status, t_win_pct, fill = plant_status)) +
  geom_col(show.legend = FALSE) + facet_wrap(~map) +
  scale_fill_manual(values = c(
    "Not planted" = "#8C9A7A",
    "Planted A" = "#5E8C8A",
    "Planted B" = "#B86F5C",
    "Planted unknown site" = "#A68B2D",
    "Unknown" = "#9A9B70"
  )) +
  geom_text(aes(label = sprintf("%.1f%% (n=%d)", t_win_pct, rounds)),
            vjust = -.3, size = 3) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
                     expand = expansion(mult = c(0, .1))) +
  labs(title = "T round-win rate by bomb-plant status", x = NULL, y = "T round wins (%)") +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))
p_plant_outcome
```




```
save_pdf(p_plant_outcome, "11_t_win_rate_by_plant_status.pdf", 9, 5)
```

5.5 Time until plant and post-plant duration

The first planted snapshot approximates the time of plant; the latest snapshot at or before `end_tick` approximates the final post-plant state. The elapsed time values are snapshot-based estimates.

```
plant_timing <- round_states_enriched |>
  filter(bomb_planted %in% TRUE) |>
  group_by(match_id, map, round_num) |>
  slice_min(tick, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(match_id, map, round_num, seconds)

post_plant_timing <- round_states_enriched |>
  filter(bomb_planted %in% TRUE, !is.na(end_tick), tick <= end_tick) |>
  group_by(match_id, map, round_num) |>
  slice_max(tick, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(match_id, map, round_num, seconds_since_plant, reason_name)
```

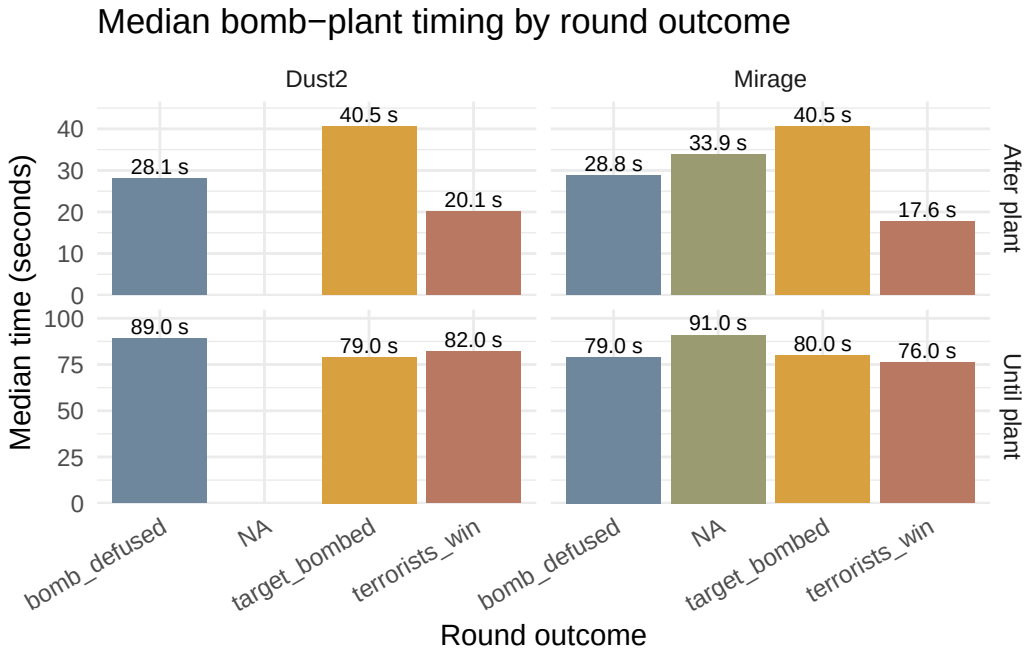
```
plant_timing_summary <- inner_join(plant_timing, post_plant_timing,
                                   by = c("match_id", "map", "round_num")) |>
  group_by(map, reason_name) |>
  summarise(
    planted_rounds = n(),
    median_seconds_until_plant = median(seconds, na.rm = TRUE),
    median_seconds_after_plant = median(seconds_since_plant, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(reason_label = if_else(is.na(reason_name), "NA", as.character(reason_name)))
show_table(plant_timing_summary |> select(map, reason_label, planted_rounds,
                                          median_seconds_until_plant,
                                          median_seconds_after_plant))
```

map	reason_label	planted_rounds	median_seconds_until_plant	median_seconds_after_plant
Dust2	bomb_defused	79	89	28.08
Dust2	target_bombed	129	79	40.48
Dust2	terrorists_win	124	82	20.09
Mirage	bomb_defused	75	79	28.83
Mirage	target_bombed	105	80	40.48
Mirage	terrorists_win	107	76	17.64
Mirage	NA	7	91	33.86

```
save_csv(plant_timing_summary |> select(map, reason_label, planted_rounds,
                                          median_seconds_until_plant,
                                          median_seconds_after_plant), "qmd_plant_timing_summ

timing_long <- plant_timing_summary |>
  select(map, reason_label, median_seconds_until_plant, median_seconds_after_plant) |>
  pivot_longer(starts_with("median_seconds"),
               names_to = "timing", values_to = "median_seconds") |>
  mutate(timing = recode(timing,
    median_seconds_until_plant = "Until plant",
    median_seconds_after_plant = "After plant"))
p_timing <- ggplot(timing_long, aes(reason_label, median_seconds, fill = reason_label)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = condition_palette, na.value = "#8C9A7A") +
  geom_text(aes(label = sprintf("%.1f s", median_seconds)), vjust = -.3, size = 2.8) +
  facet_grid(timing ~ map, scales = "free_y") +
  scale_y_continuous(expand = expansion(mult = c(0, .15))) +
  labs(title = "Median bomb-plant timing by round outcome",
```

```
       x = "Round outcome", y = "Median time (seconds)") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
p_timing
```



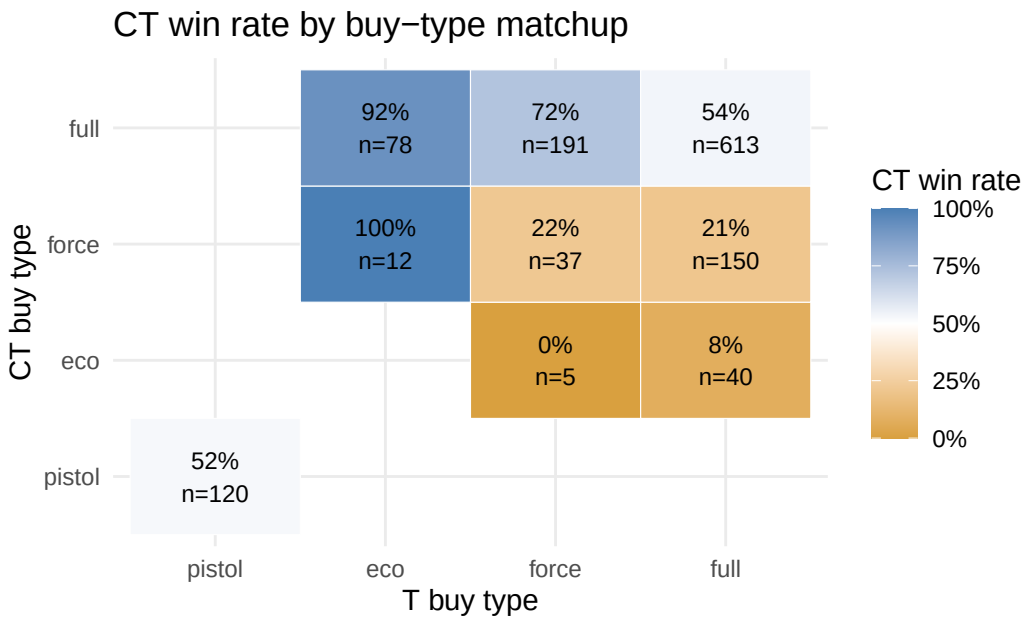

```
save_pdf(p_timing, "12_bomb_plant_timing.pdf", 11, 7)
```

5.6 CT win rate by buy-type matchup

```
buy_levels <- c("pistol", "eco", "force", "full")
buy_levels <- c(buy_levels, setdiff(unique(c(rounds$ct_buy_type, rounds$t_buy_type)),
                                    c(buy_levels, NA)))
buy <- rounds |>
  filter(!is.na(ct_buy_type), !is.na(t_buy_type)) |>
  mutate(across(c(ct_buy_type, t_buy_type), \(x) factor(x, levels = buy_levels))) |>
  group_by(ct_buy_type, t_buy_type) |>
  summarise(ct_win_rate = mean(ct_win, na.rm = TRUE), n = n(), .groups = "drop")
show_table(buy)
```

ct_buy_type	t_buy_type	ct_win_rate	n
pistol	pistol	0.52	120
eco	force	0.00	5
eco	full	0.07	40
force	eco	1.00	12
force	force	0.22	37
force	full	0.21	150
full	eco	0.92	78
full	force	0.72	191
full	full	0.54	613

```
save_csv(buy, "qmd_buy_type_matchup.csv")
p_buy <- ggplot(buy, aes(t_buy_type, ct_buy_type, fill = ct_win_rate)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.0f%%\nn=%d", 100 * ct_win_rate, n)), size = 3) +
  scale_fill_gradient2(low = "#d9a03f", mid = "white", high = "#4a7fb5",
                       midpoint = .5, limits = c(0, 1), labels = percent) +
  labs(title = "CT win rate by buy-type matchup", x = "T buy type",
       y = "CT buy type", fill = "CT win rate",
       caption = "Buy types are recorded at freeze end; missing buy types are excluded.")
p_buy
```



Buy types are recorded at freeze end; missing buy types are excluded.

```
save_pdf(p_buy, "13_buy_type_matchup_heatmap.pdf", 8, 6)
```

5.7 Round outcome versus alive-player advantage at 30 seconds

The selected row is the latest available snapshot at or before 30 seconds, not necessarily exactly at 30 seconds. One snapshot per round is used to avoid counting a round multiple times.

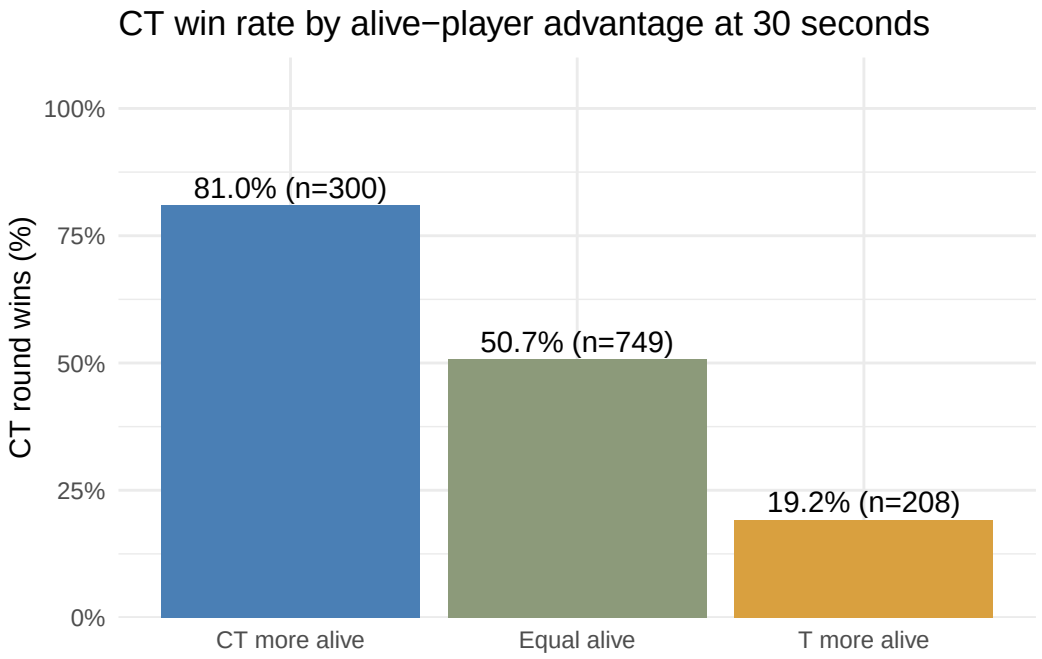
```
state_at_30s <- round_states_enriched |>
  filter(seconds <= 30) |>
  group_by(match_id, map, round_num) |>
  slice_max(seconds, n = 1, with_ties = FALSE) |>
  ungroup() |>
  mutate(alive_difference = ct_alive - t_alive,
         alive_advantage = case_when(
           alive_difference > 0 ~ "CT more alive",
           alive_difference < 0 ~ "T more alive",
           alive_difference == 0 ~ "Equal alive",
           TRUE ~ NA_character_
         ))
alive_outcome_30s <- state_at_30s |>
  filter(!is.na(ct_win), !is.na(alive_advantage)) |>
  group_by(alive_advantage) |>
  summarise(rounds = n(), ct_win_pct = 100 * mean(ct_win, na.rm = TRUE),
            .groups = "drop")
show_table(alive_outcome_30s)
```

alive_advantage	rounds	ct_win_pct
CT more alive	300	81.00
Equal alive	749	50.73
T more alive	208	19.23

```
save_csv(alive_outcome_30s, "qmd_alive_advantage_at_30s.csv")

p_alive_30 <- ggplot(alive_outcome_30s,
                     aes(alive_advantage, ct_win_pct, fill = alive_advantage)) +
  geom_col(show.legend = FALSE) +
  scale_fill_manual(values = c(
    "CT more alive" = "#4a7fb5",
    "T more alive" = "#d9a03f",
    "Equal alive" = "#8C9A7A"
```

```
  )) +
  geom_text(aes(label = sprintf("%.1f%% (n=%d)", ct_win_pct, rounds)),
            vjust = -.35) +
  scale_y_continuous(limits = c(0, 100), labels = \(x) paste0(x, "%"),
                     expand = expansion(mult = c(0, .1))) +
  labs(title = "CT win rate by alive-player advantage at 30 seconds",
       x = NULL, y = "CT round wins (%)")
p_alive_30
```




```
save_pdf(p_alive_30, "14_alive_advantage_at_30s.pdf", 8, 5)
```

5.8 Equipment, money, health, and armor advantage at 30 seconds

```
state_economy_30 <- round_states_enriched |>
  filter(seconds <= 30) |>
  group_by(match_id, map, round_num) |>
  slice_max(seconds, n = 1, with_ties = FALSE) |>
  ungroup() |>
  mutate(
    equipment_difference = ct_equipment - t_equipment,
```

```
    money_difference = ct_money - t_money,
    hp_difference = ct_hp - t_hp,
    armor_difference = ct_armor - t_armor,
    equipment_advantage = case_when(
      equipment_difference > 0 ~ "CT higher equipment",
      equipment_difference < 0 ~ "T higher equipment",
      equipment_difference == 0 ~ "Equal equipment",
      TRUE ~ NA_character_
    ),
    hp_advantage = case_when(
      hp_difference > 0 ~ "CT higher HP",
      hp_difference < 0 ~ "T higher HP",
      hp_difference == 0 ~ "Equal HP",
      TRUE ~ NA_character_
    )
  )
economy_summary <- state_economy_30 |>
  filter(!is.na(ct_win), !is.na(equipment_advantage)) |>
  group_by(equipment_advantage) |>
  summarise(rounds = n(), ct_win_pct = 100 * mean(ct_win, na.rm = TRUE),
            mean_equipment_difference = mean(equipment_difference, na.rm = TRUE),
            mean_money_difference = mean(money_difference, na.rm = TRUE),
            .groups = "drop")
show_table(economy_summary)
```

equipment_advantage	rounds	ct_win_pct	mean_equipment_difference	mean_money_difference
CT higher equipment	793	68.73	9770.11	5265.07
Equal equipment	6	50.00	0.00	-616.67
T higher equipment	458	25.11	-8162.12	-5150.87

```
save_csv(economy_summary, "qmd_equipment_advantage_summary.csv")

health_armor_summary <- state_economy_30 |>
  filter(!is.na(ct_win), !is.na(hp_advantage)) |>
  group_by(hp_advantage) |>
  summarise(rounds = n(), ct_win_pct = 100 * mean(ct_win, na.rm = TRUE),
            mean_hp_difference = mean(hp_difference, na.rm = TRUE),
            mean_armor_difference = mean(armor_difference, na.rm = TRUE),
            .groups = "drop")
show_table(health_armor_summary)
```

hp_advantage	rounds	ct_win_pct	mean_hp_difference	mean_armor_difference
CT higher HP	551	68.42	95.50	120.13
Equal HP	342	51.75	0.00	27.62
T higher HP	364	29.95	-76.66	-84.38

```
save_csv(health_armor_summary, "qmd_health_armor_advantage_summary.csv")
```

6 5. Interpretation notes and limitations

- `rounds` is the round-level source; `round_states` contains multiple snapshots per round; the enriched table combines them for analysis while retaining snapshot granularity.
- Missing planting fields can be expected when no plant occurred. In the enriched data, any missing round-level value is repeated across every snapshot for that round, which can increase the missing-cell count.
- Percentages in stacked charts are calculated within their stated grouping (usually within map); sample sizes should be considered when interpreting percentages.
- Snapshot rows from the same round are not independent observations. Analyses intended to compare round outcomes should use one selected snapshot per round, as in the 30-second and round-end sections.
- `ct_win` is the eventual round outcome repeated across snapshots; do not treat every snapshot as an independent outcome for predictive modelling. If building predictive models, split training and test data by match or round to avoid leakage.
- Round-end and bomb timing analyses use periodic snapshots and therefore approximate the exact event times.
- All figures are saved as PDFs in `../report/figures/cs2-combined_analysis`, and CSV tables are saved in `../data/analysis/cs2-combined_analysis`.